

Evaluación diagnóstica del módulo 2

PF1822 · Especialización en Desarrollo con IA

Módulo 2 · MA04576 · Introducción a modelos generativos y consumo por API

MOMENTO

Al comenzar el módulo, antes de la primera lectura

MODALIDAD

Individual, en línea: cuestionario del LMS o este documento

CALIFICACIÓN

No lleva nota: muestra tu punto de partida

VÍNCULO CON EL PLAN

La competencia del módulo, sus cuatro aprendizajes esperados y sus contenidos

Para qué sirve

Esta evaluación abre el módulo «Introducción a modelos generativos y consumo por API». No lleva nota: te muestra qué sabes hoy y qué te conviene repasar, y le muestra a tu tutor/a cómo acompañarte desde el primer día.

Las preguntas usan el caso que recorre todo el módulo: **Nube Sur**, una empresa ficticia de software que vende sistemas de gestión a pymes y cuya mesa de ayuda atiende a sus clientes por correo, formulario web y chat. Nube Sur quiere apoyarse en modelos de IA generativa para resumir tickets, clasificarlos y responder preguntas frecuentes.

Al terminar el módulo vas a ser capaz de:

Implementar la arquitectura básica de una aplicación que integre modelos de IA mediante APIs y preprocesamiento de texto para consumo por modelos de lenguaje, garantizando el consumo eficiente y la evaluación de resultados en un entorno de desarrollo de software.

El módulo se organiza en cuatro contenidos del plan, y esta evaluación tiene preguntas de cada uno:

1. Modelos generativos y ecosistema tecnológico
2. Integración y consumo de APIs
3. Diseño de prompts para APIs generativas
4. Pipeline de preprocesamiento y evaluación básica de texto

Instrucciones

- Responde sin buscar en internet ni en los materiales del curso: lo que importa es tu punto de partida real.
- En la Parte A, marca una sola opción por pregunta.

- Si no sabes una respuesta, déjala en blanco. No adivines: una pregunta en blanco también es información para ti y para tu tutor/a.
- En la Parte B, marca con honestidad cómo te ves hoy. Nadie espera que sepas lo que el módulo va a enseñarte.
- En la Parte C, responde con tus palabras, en pocas líneas.
- Si usas este documento en vez del cuestionario del LMS, guarda una copia con tus respuestas y súbela donde indique tu tutor/a.

Parte A · Lo que ya sabes

Doce preguntas, tres por cada contenido del plan. Todas parten de situaciones de Nube Sur.

Contenido 1 · Modelos generativos y ecosistema tecnológico

1. Nube Sur usa dos modelos: uno escribe el resumen de cada ticket y otro convierte cada ticket en un vector para encontrar tickets parecidos. ¿Cómo se clasifican?

- ☐ a) Los dos son modelos de difusión, porque transforman el texto de entrada en una salida nueva
- ☐ b) El primero es de difusión, porque mejora el resumen en varios pasos; el segundo es autorregresivo, porque arma el vector un número a la vez
- ☐ c) Los dos son autorregresivos: todo modelo basado en transformers genera texto token a token
- ☐ d) El primero, un transformer autorregresivo; el segundo, un transformer codificador que produce embeddings

2. Una agente de Nube Sur abre un ticket y quiere ver los tickets de la semana sobre el mismo problema, aunque estén escritos con otras palabras. ¿Qué componente de la arquitectura hace posible esa búsqueda?

- ☐ a) El modelo de IA, que recuerda los tickets que resumió antes
- ☐ b) El cliente, que guarda en el navegador los tickets abiertos en el día
- ☐ c) El vector store, que guarda los embeddings de los tickets y devuelve los más cercanos
- ☐ d) El módulo de preprocesamiento, que compara las palabras del ticket nuevo con las de los anteriores

3. Tras una caída del servicio llegan cientos de tickets casi al mismo tiempo. El servidor llama al modelo en el momento en que llega cada uno (solicitud–respuesta) y el proveedor empieza a responder con el código 429. ¿Qué cambio resuelve mejor este problema de escalabilidad?

- ☐ a) Que cada agente llame al modelo desde su navegador con su propia clave, para repartir el límite
- ☐ b) Encolar los tickets en el servidor y procesarlos por lotes, al ritmo que permite el proveedor
- ☐ c) Reintentar de inmediato cada solicitud rechazada hasta que el proveedor la acepte, para no perder ningún ticket
- ☐ d) Cambiar a un modelo más grande del mismo proveedor, porque procesa más texto en cada llamada

Contenido 2 · Integración y consumo de APIs

4. Enviaste tres textos en el campo `input` del endpoint de embeddings de OpenAI. Según la documentación oficial, ¿dónde viene el vector del segundo texto?

- ☐ a) En el elemento de `data` cuyo `index` es 1, en su campo `embedding`
- ☐ b) En `choices[1].message.content`, igual que en el endpoint de chat
- ☐ c) En `usage.total_tokens`, que resume el vector en un solo número
- ☐ d) En `data[0].embedding[1]`: `data` trae un solo objeto y su `embedding` guarda los tres vectores

5. Al arrancar, el servidor de Nube Sur comprueba que su clave funciona pidiendo la lista de modelos disponibles. ¿Qué solicitud es la adecuada?

- ☐ a) Un POST a `/v1/models` con la clave en el cuerpo JSON, porque toda llamada a un modelo es POST
- ☐ b) Un GET a `/v1/models` con la clave como parámetro de la URL (`?api_key`), para no tener que armar encabezados
- ☐ c) Un GET a `/v1/models` con la clave escrita en una constante del código, porque el repositorio es privado
- ☐ d) Un GET a `/v1/models` con la clave leída de una variable de entorno, en el encabezado `Authorization: Bearer`

6. Nube Sur llama al mismo modelo para resumir tickets (`temperature` 0,2 y 60 tokens) y para redactar borradores de respuesta (`temperature` 0,7 y 300 tokens). ¿Qué diseño es más fácil de mantener y de probar?

- ☐ a) Copiar la función en dos archivos, uno por tarea, cada uno con su URL, su clave y su manejo de errores
- ☐ b) Una sola función con `temperature` y `max_tokens` fijos, que se editan a mano antes de cada uso
- ☐ c) Una clase con un método `generar(mensajes, temperature, max_tokens)` que concentra URL, clave y errores
- ☐ d) Escribir la solicitud HTTP completa en cada lugar del código donde haga falta, para que cada tarea se lea sola, sin saltar a otro archivo

Contenido 3 · Diseño de prompts para APIs generativas

7. El resumidor de Nube Sur funciona bien con los tickets que reportan fallas, pero los mensajes que solo agradecen ("¡Gracias, ya funciona!") los resume como si fueran fallas nuevas. ¿Qué cambio ayuda más?

- ☐ a) Subir `temperature`, para que el modelo se adapte mejor a cada tipo de ticket y no repita siempre el mismo resumen
- ☐ b) Agregar dos ejemplos resueltos (few-shot), uno de ellos un agradecimiento con su resumen esperado

- ☐ c) Quitar el mensaje de sistema, para que el modelo no se confunda con tantas instrucciones
- ☐ d) Agregar al final del prompt "esfuérzate y piensa bien antes de resumir", para que ponga más atención

8. Un programa lee la salida del resumidor para mostrarla en el tablero de la mesa de ayuda. ¿Qué mensaje de sistema es el más adecuado?

- ☐ a) "Eres analista de soporte. Devuelve solo JSON con `resumen` (máximo 25 palabras) y `prioridad` (alta, media o baja)."
- ☐ b) "Resume el ticket de forma breve y clara para el tablero, y di al final qué tan urgente te parece."
- ☐ c) "Eres el mejor analista del mundo. Resume el ticket con todo el detalle posible, explica tu razonamiento paso a paso y ordénalo en párrafos."
- ☐ d) "Resume el ticket en formato JSON para que el tablero pueda leerlo, con los campos que te parezcan útiles."

9. Algunos resúmenes salen cortados a la mitad y la respuesta de la API trae `"finish_reason": "length"`. ¿Qué ajuste corresponde en la siguiente iteración?

- ☐ a) Bajar `temperature` a 0, para que el modelo elija palabras más cortas y termine antes
- ☐ b) Bajar todavía más `max_tokens` y agregar "sé breve", para forzar un resumen corto
- ☐ c) Ajustar `top_p` y `temperature` a la vez, para controlar mejor el largo
- ☐ d) Subir `max_tokens` con margen y pedir el largo en el prompt: "máximo 25 palabras"

Contenido 4 · Pipeline de preprocesamiento y evaluación básica de texto

10. En el ticket "No puedo descargar las facturas", ¿cuál de estas afirmaciones es correcta?

- ☐ a) El lema de "facturas" es "factura", y "descargar las" es un bigrama
- ☐ b) El lema de "puedo" es "pued", y "no puedo descargar" es un bigrama
- ☐ c) "facturas" es una stopword, porque aparece en muchos tickets de Nube Sur
- ☐ d) La frase tiene cuatro tokens, porque las stopwords no se cuentan como tokens

11. Un ticket llega así: `<p>Hola!!! Desde ayer el módulo de inventario da "error 500" al guardar un producto.</p><p>Saludos, Rosa Díaz . rosa.diaz@ferreteria.test</p>`. ¿Qué limpieza conviene antes de enviarlo al modelo?

- ☐ a) Ninguna: el modelo ignora el HTML y el correo le da contexto útil para responder
- ☐ b) Quitar etiquetas y firma con nombre y correo, dejar "!!!" en "!" y conservar "error 500"
- ☐ c) Borrar todo lo que no sea letra, incluidos números y comillas, para dejar solo palabras
- ☐ d) Pasar todo a minúsculas y quitar las tildes, porque así el modelo lo entiende mejor y gasta menos tokens

12. Vas a comparar dos versiones del prompt del resumidor con ROUGE, BLEU, coherencia y relevancia. ¿Qué forma de registrar los resultados te permite decidir con evidencia?

- ☐ a) Anotar solo el mejor resultado de cada versión, para que la tabla quede corta
- ☐ b) Probar cada versión con tickets distintos, para cubrir más casos con el mismo esfuerzo
- ☐ c) Los mismos tickets, referencias y tokenizador en las dos versiones, con una fila por prueba

en un CSV

☐ d) Leer las salidas y anotar la impresión general de cada versión, porque las métricas no captan el sentido del resumen

Parte B · Cómo te ves hoy frente a los aprendizajes del módulo

Para cada criterio de evaluación del plan, marca el nivel que mejor te describe hoy:

Nivel	Significa	Por ejemplo, frente a "llamar a una API desde Python"
1	Aún no lo sé hacer	No sabría por dónde empezar.
2	Lo hago con ayuda	Lo hago siguiendo un tutorial o con alguien al lado.
3	Lo hago solo/a, con dudas	Lo hago, pero cuando falla no sé bien por qué.
4	Lo hago solo/a y puedo explicarlo	Lo hago y podría explicárselo a un compañero/a, incluido qué hacer cuando falla.

Aprendizaje esperado 1: Analizar el rol de los modelos generativos en el desarrollo de aplicaciones, describiendo sus tipos, casos de uso, tecnologías asociadas y representando arquitecturas funcionales que integren estos modelos mediante APIs.

Criterio de evaluación	1	2	3	4
1.1 Clasifica los tipos de modelos generativos identificando sus características y aplicaciones más frecuentes en la organización.				
1.2 Representa un diagrama funcional de una aplicación con IA generativa, explicando la función de cada componente dentro de la arquitectura.				
1.3 Analiza la elección de la arquitectura en función de criterios como escalabilidad, seguridad o mantenimiento.				

Aprendizaje esperado 2: Utilizar un modelo de IA a través de API REST utilizando solicitudes HTTP y autenticación, basándose en la documentación oficial de OpenAI y Hugging Face.

Criterio de evaluación	1	2	3	4
2.1 Explica la estructura y principales secciones de los endpoints de generación y embeddings de acuerdo con la documentación oficial de OpenAI y Hugging Face.				
2.2 Implementa llamadas HTTP GET y POST a endpoints de completions y embeddings para el consumo de servicios, de acuerdo con la documentación oficial.				
2.3 Implementa una función o clase modular en Python que encapsula la llamada al API y permite reutilizarla con distintos parámetros, documentada con docstrings y pruebas unitarias simples.				

Aprendizaje esperado 3: Formular prompts efectivos para modelos generativos en escenarios zero-shot o few-shot, considerando la tarea solicitada y los parámetros de generación disponibles.

Criterio de evaluación	1	2	3	4
3.1 Explica la función de un prompt en la interacción con modelos generativos, distinguiendo entre estrategias zero-shot y few-shot, e identificando sus ventajas y limitaciones básicas.				
3.2 Elabora prompts adecuados para tareas específicas, como resumen, generación de código o reformulación de texto, seleccionando la estrategia de prompting más apropiada según el objetivo.				
3.3 Ajusta un prompt de forma iterativa con base en los outputs obtenidos, describiendo los cambios realizados en términos de claridad, estructura o control del comportamiento del modelo.				

Aprendizaje esperado 4: Implementar un pipeline de preprocesamiento y evaluación básica de texto en Python, aplicando técnicas de limpieza, normalización y métricas iniciales de calidad, de acuerdo con los requisitos de entrada y salida del modelo.

Criterio de evaluación	1	2	3	4
4.1 Explica los conceptos fundamentales de NLP y evaluación básica de respuestas, considerando token, lema, stopwords, n-gramas, coherencia y relevancia.				
4.2 Codifica una rutina en Python para la limpieza de texto, tokenización y normalización sobre un corpus de documentos, de acuerdo con buenas prácticas.				
4.3 Calcula métricas básicas de calidad de respuestas generadas, considerando coherencia, relevancia, BLEU y ROUGE, registrando los resultados obtenidos de acuerdo con un protocolo definido.				

Guarda tus respuestas de esta parte. Al cierre del módulo las vas a copiar en la columna «Al inicio» de la autoevaluación, y las vas a usar en la introducción y en el cierre de tu portafolio para mostrar cuánto avanzaste. Si respondes en el LMS, anota también tus niveles en tu bitácora o en un archivo propio.

Parte C · Tu experiencia

1. ¿Has usado alguna herramienta de IA generativa (para texto, código o imágenes) en tu trabajo o en tus estudios? Cuenta para qué la usaste y qué te costó controlar de sus respuestas.
2. ¿Has llamado a una API desde código o con un cliente HTTP como Postman? Describe qué hiciste y dónde guardabas la clave o el token.
3. ¿Has escrito pruebas unitarias, usado entornos virtuales de Python o trabajado con Git y GitHub? Indica qué has hecho y con qué frecuencia.
4. Piensa en una mesa de ayuda o un servicio de atención que conozcas. ¿Qué preguntas repetidas le dejarías responder a un asistente con IA, cuáles no, y por qué?

Qué pasa después

- Tu tutor/a revisa tus respuestas y te devuelve tus aciertos de la Parte A por contenido (de 0 a 3) y un comentario breve: en qué contenido conviene reforzar, repasar o avanzar, y con

qué recurso del módulo. Guárdalos junto con tu Parte B: los vas a usar en la introducción de tu portafolio.

- Con tus respuestas de la Parte C, tu tutor/a arma los equipos de los ABPRO para que cada uno reúna experiencias distintas.
- Empiezas por la lectura del primer contenido, «Modelos generativos y su ecosistema».
- Al cierre del módulo vuelves a esta evaluación: tu Parte B es el punto de comparación de la autoevaluación y de tu portafolio.